FastAPI Todo App

Explained from zero: database.py, models.py and main.py, in plain language

1. The big picture

What you have is a tiny **web API**. An API is just a program that sits and waits. Someone (your browser, a phone app, another program) sends it a message like *"give me all todos"* or *"add this todo"*, and the API sends an answer back.

Your API can do four things with todos: **C**reate, **R**ead, **U**pdate, **D**elete. People call this **CRUD**. Almost every backend you will ever build is CRUD plus extra stuff on top.

> **Think of a restaurant**
> You (the client) tell the **waiter** (FastAPI) what you want. The waiter first checks your order makes sense (validation), then walks to the **storage room** (database) through a **translator** (SQLAlchemy) who speaks the storage room's language (SQL). Then the waiter brings the result back to you.

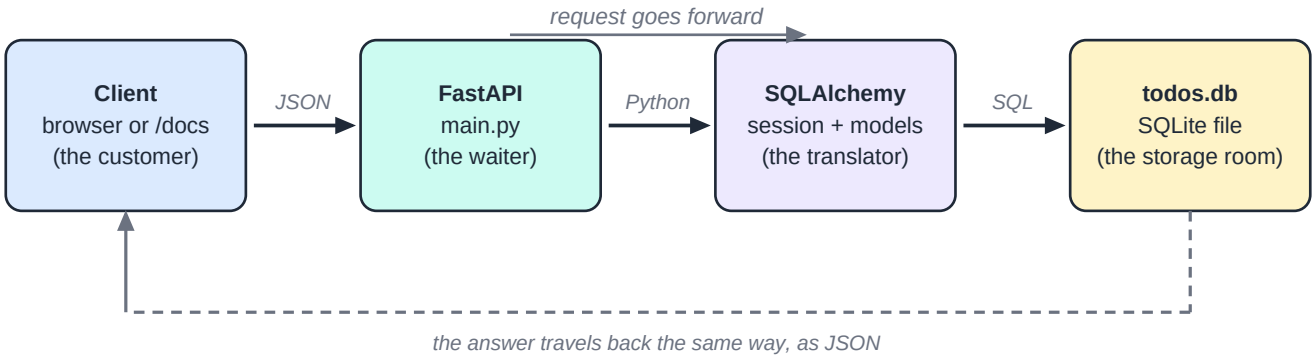

Figure 1: the journey of every request

2. Three files, three jobs

File	Its one job	Analogy
`database.py`	Sets up the connection to the database	Plugging in the cable
`models.py`	Describes what the *todos table* looks like (its columns)	Drawing the empty spreadsheet
`main.py`	The actual API: URLs, rules, and the code that runs for each URL	The waiter and the menu

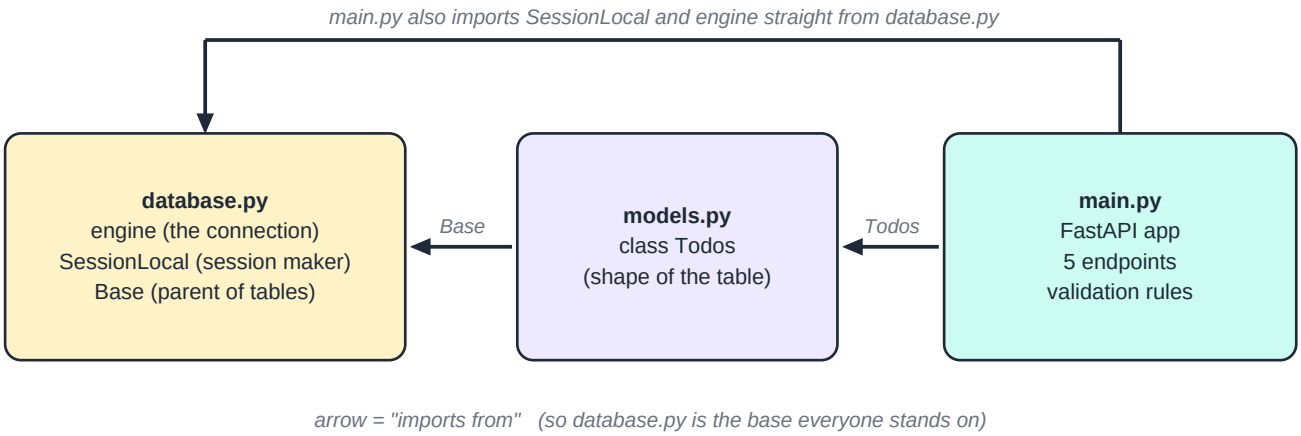

Figure 2: who imports whom. database.py depends on nobody, so we read it first.

> **Reading order**
> Always read in this order: **database.py** then **models.py** then **main.py**. Each file only makes sense after the one before it.

3. database.py: plugging in the database

```
SQLALCHEMY_DATABASE_URL = 'sqlite:///./todos.db'
engine = create_engine(SQLALCHEMY_DATABASE_URL,
                       connect_args={'check_same_thread': False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

Line	In plain words
`SQLALCHEMY_DATABASE_URL = 'sqlite:///./todos.db'`	The **address** of your database. **sqlite** is the type. **./todos.db** means "a file called todos.db in this same folder". SQLite is just one file on your disk, so there is nothing to install or run. Later you can swap this one line for a Postgres address and most of the code stays the same.
`engine = create_engine(...)`	The **cable plugged into the database**. It knows where the database is and how to talk to it. It does not send anything by itself, it is just the connection.
`check_same_thread: False`	SQLite normally says "only the thread that opened me may use me". FastAPI handles requests on different threads, so this switch turns that rule off. You need it for SQLite + FastAPI, and you can forget about it after that.
`SessionLocal = sessionmaker(...)`	A **machine that makes sessions**. A session is one conversation with the database. Think of it as a **shopping cart**: you put changes in it, and nothing is permanent until you press Pay (*commit*).
`autocommit=False`	"Do not save by yourself. I will tell you when, using db.commit()."
`autoflush=False`	"Do not quietly push my half-finished changes to the database before I ask."
`bind=engine`	"Use this cable (engine) to talk to the database."

Line	In plain words
`Base = declarative_base()`	The **parent class** for all your tables. Every table class in models.py will inherit from it. SQLAlchemy keeps a list of everything that inherits from Base, and that is how `create_all` later knows which tables to build.

> **Tiny heads-up**
> In SQLAlchemy 2.x the recommended import is `from sqlalchemy.orm import declarative_base`. The `sqlalchemy.ext.declarative` one in your file still works, it may just print a deprecation warning. Not a bug.

4. models.py: drawing the table

```
class Todos(Base):
    __tablename__ = 'todos'

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    description = Column(String)
    priority = Column(Integer)
    complete = Column(Boolean, default=False)
```

The rule is simple: **one class = one table**, **one Column line = one column**, and later **one object = one row**. You are writing the table in Python instead of SQL, and SQLAlchemy converts it for you.

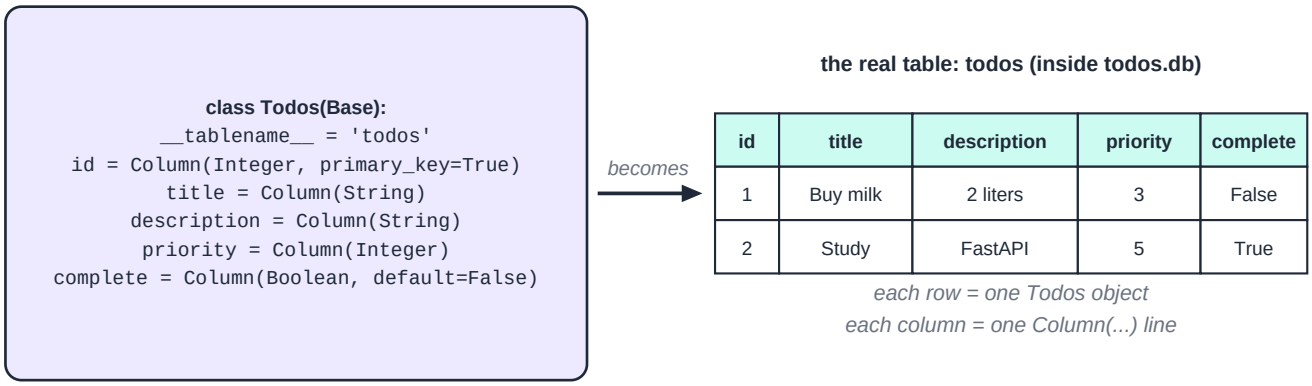

Figure 3: a Python class on the left, the real database table on the right

Piece	In plain words
`class Todos(Base)`	Inherits from Base, so SQLAlchemy knows "this class is a table".
`__tablename__ = 'todos'`	The name the table gets inside todos.db.
`primary_key=True`	This column is the **unique ID** of each row. The database fills it in itself: 1, 2, 3, ... You never set id by hand.
`index=True`	Builds a lookup shortcut, like the index at the back of a book. Searching by id gets faster.
`Column(String / Integer / Boolean)`	The type of data allowed in that column: text, whole number, True/False.
`default=False`	If nobody says whether a todo is complete, it starts as False.

Do not mix these two up (it matters in the next section)

`Todos` in models.py = the **table** (what is stored). `TodoRequest` in main.py = the **rules for incoming data** (what the client is allowed to send). Two different jobs, two different classes.

5. main.py, part 1: the setup

Imports in one glance

`FastAPI`	The class that creates your app.
`Depends`	Says "before running this function, first run that other function and give me its result".
`Annotated`	Lets you attach extra info to a type. Used here to build a reusable shortcut.
`HTTPException`	Lets you stop and send an error back (like 404 Not Found).
`Path`	Adds rules to a value that sits inside the URL (like the id).
`status`	A list of named status codes, so you write status.HTTP_201_CREATED instead of a bare 201.
`BaseModel, Field`	From Pydantic. Used to describe and validate incoming data.
`Session`	The type of a database session. Used only so your editor can help you.

```
app = FastAPI()

models.Base.metadata.create_all(bind=engine)
```

- `app = FastAPI()` creates your application. Everything below hangs on this `app` object, and it is what the server runs.
- `create_all(bind=engine)` runs **once when the app starts**. It looks at every class that inherits from Base (your Todos) and creates the matching table in todos.db **if it does not exist yet**. That is why a todos.db file appears the first time you run the app.

Gotcha you will hit one day

`create_all` only **creates missing tables**. If you later add a column to Todos, it will **not** change the existing table. While learning, just delete todos.db and restart. In real projects you use a migration tool called Alembic.

get_db: lending out a database session

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Every request needs its own session (its own shopping cart), and every session must be closed when the request is done, otherwise connections pile up and your app slows down or breaks. This function does exactly that.

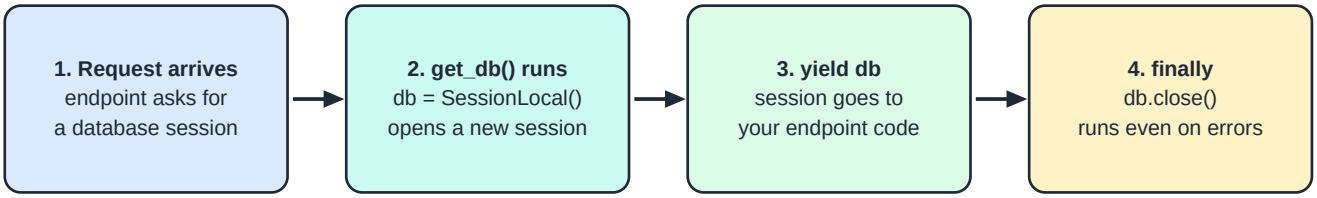

Figure 4: the life of a session during one request

`yield` is like `return` but the function **pauses** instead of ending. FastAPI takes the session, runs your endpoint code, and when the endpoint is finished it comes back and continues after the yield, which runs `finally: db.close()`. Because of `finally`, the session closes even if your endpoint crashed halfway.

The shortcut line

```
db_dependency = Annotated[Session, Depends(get_db)]
```

Without this, every endpoint would need to write `db: Session = Depends(get_db)`. This line packs that into one reusable name. So when you see `db: db_dependency` in an endpoint, read it as: **"FastAPI, run get_db and hand me the session as db."** You never call get_db yourself.

TodoRequest: the bouncer at the door

```
class TodoRequest(BaseModel):
    title: str = Field(min_length=3)
    description: str = Field(min_length=3, max_length=100)
    priority: int = Field(gt=0, lt=6)
    complete: bool
```

This class lists **what a valid todo looks like when a client sends it to you**. FastAPI checks every incoming JSON body against it **before** your function even starts.

Field	Rule
`title: str = Field(min_length=3)`	Must be text, at least 3 characters.
`description ... max_length=100`	Text between 3 and 100 characters.
`priority: int = Field(gt=0, lt=6)`	Whole number, **g**reater **t**han 0 and **l**ess **t**han 6. So 1 to 5.
`complete: bool`	Must be true or false. No default, so the client has to send it.

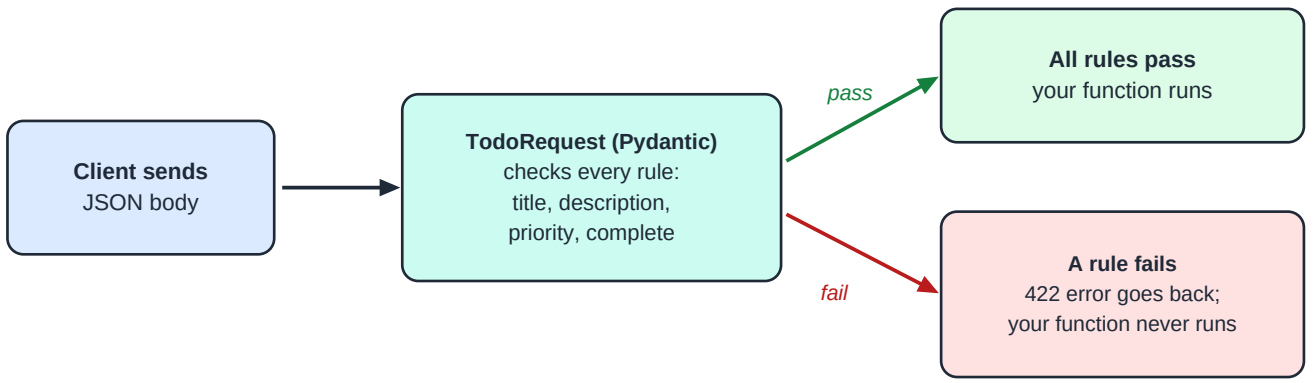

Figure 5: validation happens before your code runs. You wrote zero if-statements for this.

6. main.py, part 2: the endpoints

An **endpoint** is one URL plus one action. The line starting with `@app.get(...)` is called a **decorator**. Read it as: *"when someone sends a GET request to this URL, run the function right below me."*

The four HTTP methods you use, in simple words: **GET** = "give me", **POST** = "here is something new, store it", **PUT** = "replace that with what I am sending", **DELETE** = "remove it".

Method	URL	Function	What it does	Success code
GET	`/`	read_all	Return every todo	200
GET	`/todo/{todo_id}`	read_table	Return one todo by id	200
POST	`/todo`	create_todo	Add a new todo	201
PUT	`/todo/{todo_id}`	update_todo	Replace an existing todo	204
DELETE	`/todo/{todo_id}`	delete_todo	Remove a todo	204

Two things that appear in every endpoint:

- `async def` lets FastAPI run the function without blocking others. (There is a small catch with databases, see the last section.)
- `status_code=...` is the number sent back with the response so the client knows how it went.

GET / : read_all

```
@app.get("/", status_code=status.HTTP_200_OK)
async def read_all(db: db_dependency):
    return db.query(Todos).all()
```

- `db: db_dependency` : FastAPI gives you a session here (via get_db).
- `db.query(Todos).all()` : "from the todos table, give me every row." In SQL this is `SELECT * FROM todos`.
- The list of Todos objects is turned into JSON automatically and sent back.

GET /todo/{todo_id} : read_table

```
@app.get("/todo/{todo_id}", status_code=status.HTTP_200_OK)
async def read_table(db: db_dependency, todo_id: int = Path(gt=0)):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is not None:
        return todo_model
    raise HTTPException(status_code=404, detail='Todo not found.')
```

- `{todo_id}` in the URL is a **blank to fill in**. Visiting `/todo/5` makes todo_id = 5. Because of `: int`, FastAPI converts the text "5" to the number 5 (and rejects `/todo/abc`).
- `Path(gt=0)` : the id must be above 0, so `/todo/0` and `/todo/-3` are rejected for free.
- `.filter(Todos.id == todo_id)` : "keep only the row whose id matches."
- `.first()` : "give me the first match, or None if nothing matched." Together, SQL: `SELECT * FROM todos WHERE id = 5 LIMIT 1`.
- If we found it, return it. If not, `raise HTTPException(404)` stops the function and sends a 404 error.

POST /todo : create_todo

```
@app.post("/todo", status_code=status.HTTP_201_CREATED)
async def create_todo(db: db_dependency, todo_request: TodoRequest):
    todo_model = Todos(**todo_request.model_dump())

    db.add(todo_model)
    db.commit()
```

This is the most important one to understand, so here is the whole journey of a POST:

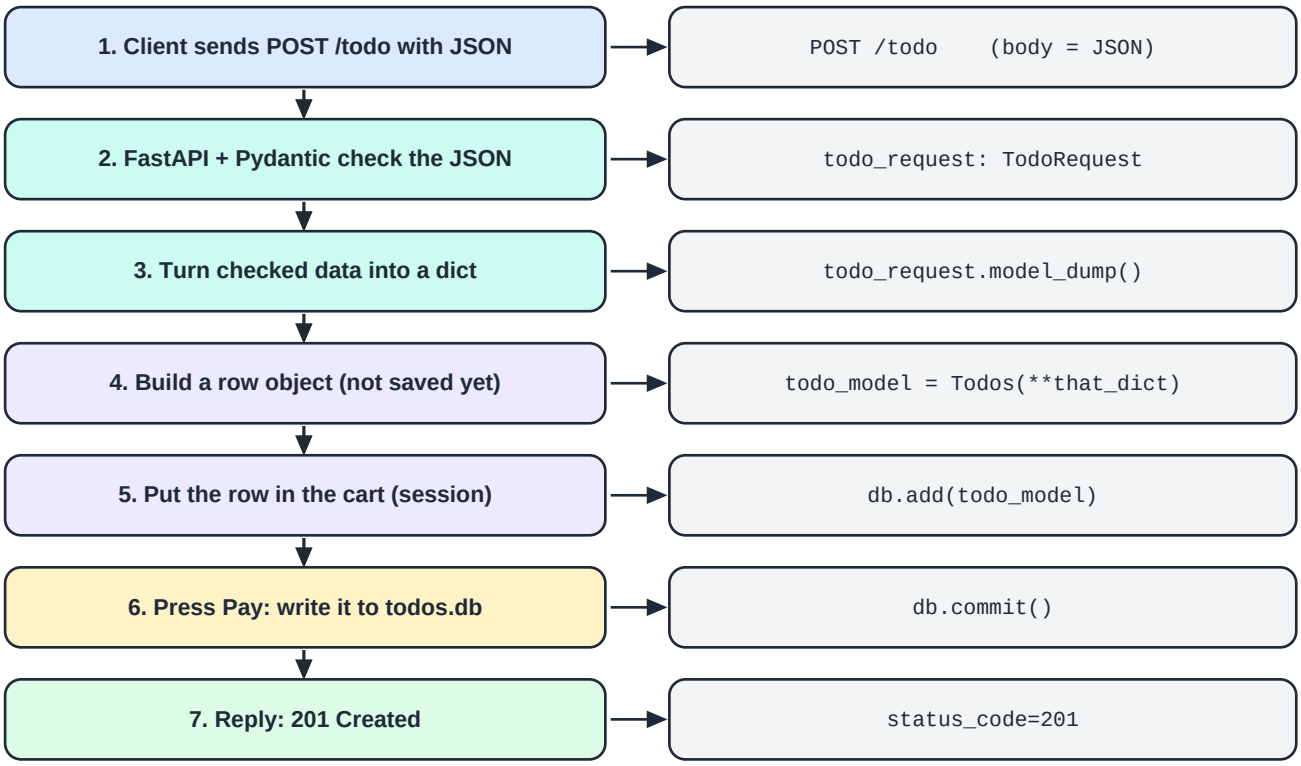

Figure 6: what happens, step by step, when a client creates a todo

- `todo_request: TodoRequest` : FastAPI reads the JSON the client sent, checks it against the rules, and gives you a clean object. If the rules fail, your function never starts.
- `model_dump()` : turns that object into a plain dictionary, like `{'title': 'Buy milk', 'priority': 3, ...}`.
- `Todos(**the_dict)` : the double star **unpacks** the dictionary into keyword arguments. This is the same as writing `Todos(title='Buy milk', priority=3, ...)` by hand, just shorter.
- Notice there is **no id**. The database creates it when the row is saved.
- `db.add(...)` puts the new row in your cart. **Nothing is saved yet.**
- `db.commit()` presses Pay. Now it is really written into todos.db.

PUT /todo/{todo_id} : update_todo

```
@app.put("/todo/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
async def update_todo(db: db_dependency,
                      todo_request: TodoRequest,
                      todo_id: int = Path(gt=0)):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')

    todo_model.title = todo_request.title
    todo_model.description = todo_request.description
    todo_model.priority = todo_request.priority
    todo_model.complete = todo_request.complete

    db.add(todo_model)
    print("item updated")
    db.commit()
```

- **Step 1, find it:** same lookup as the GET one. If it does not exist, send 404.
- **Step 2, change it:** copy each new value from the request onto the row object. At this point only the Python object changed, not the database.
- **Step 3, save it:** `db.commit()` writes the changes to the file.
- PUT means **replace the whole thing**, so the client must send all four fields, not just the one it wants to change.
- **204 No Content** means "worked, and I have nothing to send back".
- The `print("item updated")` shows up in the terminal where uvicorn is running, **not** in the response the client sees.
- Extra: `db.add(todo_model)` is not actually needed here, because the session already tracks an object it loaded itself. It does no harm, it is just a habit.

DELETE /todo/{todo_id} : delete_todo

```
@app.delete("/todo/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_todo(db: db_dependency, todo_id: int = Path(gt=0)):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')
    db.query(Todos).filter(Todos.id == todo_id).delete()

    db.commit()
```

- Find it first. If it is missing, 404. (This check is why you get a clear error instead of silent nothing.)
- `.delete()` on the query removes the matching row. In SQL: `DELETE FROM todos WHERE id = 5`.
- `db.commit()` makes it permanent. Forget the commit and the row comes back, because the cart was never paid for.
- A shorter way to write the same thing is `db.delete(todo_model)`, since you already have the row in hand.

7. The SQL behind the code

You never typed a single line of SQL in this project, but SQL is running the whole time. SQLAlchemy is an **ORM** (Object Relational Mapper): you write normal Python, and it **translates** your Python into SQL and sends that SQL to the database.

Figure 7: the same request, in Python on the left and in SQL on the right

Every line in your app, translated

Your Python	The SQL it runs
`models.Base.metadata.create_all(...)`	`CREATE TABLE todos (id INTEGER PRIMARY KEY, title VARCHAR, description VARCHAR, priority INTEGER, complete BOOLEAN);`
`db.query(Todos).all()`	`SELECT * FROM todos;`
`db.query(Todos).filter(Todos.id == 5).first()`	`SELECT * FROM todos WHERE id = 5 LIMIT 1;`
`db.add(Todos(...))` `db.commit()`	`INSERT INTO todos (title, description, priority, complete) VALUES ('Buy milk', '2 liters', 3, 0);`

`todo_model.title = ...`
`(change fields)`
`db.commit()`
	`UPDATE todos SET title = '...', description = '...', priority = 4, complete = 1 WHERE id = 5;`

`db.query(Todos).filter(Todos.id == 5).delete()` `db.commit()`	`DELETE FROM todos WHERE id = 5;`

Watch it happen live

Change one line in database.py: `create_engine(URL, connect_args={...}, echo=True)`. Now every SQL statement SQLAlchemy sends is printed in your terminal as you click around in /docs. This is the fastest way to connect the Python to the SQL.

SQLite stores True/False as 1/0

That is why the INSERT above shows `0` for complete. SQLite has no real boolean type, SQLAlchemy converts for you.

SQL cheat sheet: 6 words that cover 90% of daily use

Word	Meaning in plain words
`SELECT`	Read data. *"Show me..."*
`INSERT`	Add a new row.
`UPDATE`	Change existing rows.
`DELETE`	Remove rows.
`FROM table`	Which table to use.
`WHERE`	Filter: only the rows that match. **Without it, the command hits every row.**

> **The classic beginner disaster**
> `DELETE FROM todos;` or `UPDATE todos SET complete = 1;` with **no WHERE** deletes or changes **every single row**. Always write the WHERE first, then the rest.

Practice queries (run these in DB Browser for SQLite)

Add a few todos through /docs first, then open todos.db in DB Browser for SQLite, go to the **Execute SQL** tab, and run these. The right column shows the same thing in SQLAlchemy.

SQL (run this)	Same thing in SQLAlchemy
`SELECT * FROM todos;`	`db.query(Todos).all()`
`SELECT title, priority FROM todos;`	`db.query(Todos.title, Todos.priority).all()`
`SELECT * FROM todos WHERE complete = 0;`	`db.query(Todos).filter(Todos.complete == False).all()`
`SELECT * FROM todos WHERE priority >= 4 AND complete = 0;`	`db.query(Todos).filter(Todos.priority >= 4, Todos.complete == False).all()`
`SELECT * FROM todos ORDER BY priority DESC;`	`db.query(Todos).order_by(Todos.priority.desc()).all()`
`SELECT COUNT(*) FROM todos WHERE complete = 1;`	`db.query(Todos).filter(Todos.complete == True).count()`
`SELECT * FROM todos WHERE title LIKE '%milk%';`	`db.query(Todos).filter(Todos.title.like('%milk%')).all()`
`SELECT * FROM todos LIMIT 3;`	`db.query(Todos).limit(3).all()`
`UPDATE todos SET complete = 1 WHERE id = 2;`	`todo.complete = True` `db.commit()`
`DELETE FROM todos WHERE id = 2;`	`db.query(Todos).filter(Todos.id == 2).delete()` `db.commit()`

Mini challenge: before you run each query, guess how many rows it will return. Then run it and check. That one habit builds SQL intuition faster than reading.

8. Status codes you just met

Code	Name	Meaning in plain words	Where it appears
200	OK	Worked, and here is your data	The two GET endpoints
201	Created	Worked, and something new now exists	POST
204	No Content	Worked, nothing to send back	PUT and DELETE
404	Not Found	That id does not exist	You raise it yourself
422	Unprocessable	Your data broke the rules	FastAPI sends it automatically

9. Run it and poke it

This is where it clicks. Do these in order:

1	Install: `pip install fastapi uvicorn sqlalchemy`
2	In the folder with the three files, run: `uvicorn main:app --reload`
3	Open `http://127.0.0.1:8000/docs` in your browser. FastAPI built this page for you (Swagger UI).
4	Click **POST /todo**, then **Try it out**, paste the JSON below, then **Execute**.
5	Now try GET /, GET /todo/1, PUT, DELETE. Watch the responses.
6	Break the rules on purpose: set priority to 9 or title to "ab". Read the 422 error.

```
{
  "title": "Learn FastAPI",
  "description": "Finish the todo app",
  "priority": 4,
  "complete": false
}
```

`main:app` means: file `main.py`, variable `app`. `--reload` restarts the server whenever you save a file. After the first run you will see `todos.db` appear in the folder. You can open it with the free *DB Browser for SQLite* app and see your rows with your own eyes.

10. Things to know and things to try

Honest rough edges in this code (normal for a first app)

- **POST returns nothing.** The client never learns the new todo's id. Common upgrade: call `db.refresh(todo_model)` after the commit and `return todo_model`.
- **async def with a normal database.** The DB calls here are blocking. It is fine for learning and small apps. On a busy app, plain `def` endpoints are safer because FastAPI then runs them in a thread pool.
- **No users or login.** Anyone who can reach the API can edit anything. Authentication is the usual next topic.
- **Changing columns** needs a fresh todos.db (or Alembic), as covered in section 5.

Try these to prove you understood

- Make POST return the created todo, including its new id.
- Add a new endpoint `GET /todo/priority/{priority}` that returns only todos of that priority. Hint: it is the same pattern as GET one, but with `.filter(Todos.priority == priority).all()`.
- Add a `due_date` column to Todos. Delete todos.db first, then restart. Why was deleting needed?
- Change the max description length to 200 in TodoRequest and test it in /docs.

> **The whole app in five sentences**
>
> database.py makes the connection and the session maker. models.py describes the todos table. main.py creates the table, lends each request a session through get_db, and checks incoming data with TodoRequest. Each endpoint finds or builds a Todos row using that session. Nothing reaches the file until db.commit().